

Pawgress - Team13

Pawgress - Team13 (*student proposal*)

Assessment: ♦ Fit with adjustments · **Category:** Games · **Assessed:** 2026-09-21 · **Status:** awaiting instructor approval

Proposal (as submitted): *Pawgress* is a wellness app that turns everyday walking into play: your steps help a **virtual pet** grow and stay happy, while your walks gradually reveal areas of Singapore on an **interactive map**. Set daily walking goals, build a streak, explore new places, and earn rewards to care for and customise your pet. Named technical features: **device step-counter synchronisation**; **GPS-based map exploration** (detect explored areas, update map progress); a **virtual-pet progression system** (walking + milestones → pet EXP, growth, rewards); and **offline save & sync** (store progress locally offline, sync to backend when reconnected). Loop: a local C++ engine handling rendering logic and loops, the glue between app and backend, data transfer to/from the backend, and update loops for local UI + rendered graphics (the pet avatar). Tools: three **debugging tools** — a GPS spoofer, a simulated-tracking-data injector (steps, distance), and a network-break inspector (client-sent vs server-accepted). Design features: GPS map exploration; real step tracking; virtual-pet progression & customisation; and daily goals & streaks.

Note on category: filed under **Games** — Pawgress is a **gamified-wellness app** (virtual-pet progression, rewards, streaks, exploration), the exact shape the module's Games design pool covers under its gamified-apps lens (Duolingo / Forest / Habitica / Healthy 365 are its benchmarks). Not Visualization & Analytics (it's a pet/exploration game, not a data dashboard) and not Social Media (no social core as written).

Why "with adjustments" (conditions for week-5 approval):

1. **Add a content-authoring tool** as the graded Custom Tool. The three listed tools (GPS spoofer, tracking-data injector, network-break inspector) are excellent **debugging infrastructure** — keep them (they live under G3 ★) — but the module's Tools requirement expects a **content-authoring** tool. Add

one, e.g. an editor to author the **map exploration regions / Singapore landmarks**, the **pet growth stages + customisation items**, and the **reward / milestone / streak rules** as data (this becomes M2G2 ★, and it directly serves the "later support other workstations / more content" goal).

2. **Keep the C++ loop and 2D rendering load-bearing** (avoid CRUD collapse). As written, the risk is that the app becomes a Firebase-backed step counter with a picture of a pet. To be a real module project, the C++ must do real work: an **animated, simulated pet** (growth/mood/reaction states + reward VFX), a **custom-rendered pan/zoom map of Singapore with on-device fog-of-war exploration** computed from GPS, and **real offline-sync merge/conflict logic** — with pet/region/reward definitions data-driven. The step/GPS/Firebase sync is the plumbing, not the deliverable.

With those two in place this is a solid Games (gamified-wellness) project; the pieces (data-driven pet sim, custom 2D map + fog-of-war, offline-sync, a content editor) are all genuine module work.

References:

- [Pokémon GO — Wikipedia](#) (the benchmark **GPS walk-to-explore** mobile game: walking the real world reveals and unlocks map content — the GPS-map-exploration precedent)
- [Walkr — Gamified Fitness Walk \(App Store\)](#) (a gamified step-tracker where your **steps drive progression and exploration** through a playful world — the steps-to-progression + explore precedent)
- [Wokamon — Monster Walk Quest \(official\)](#) (a **virtual pet fed and grown by your steps**, unlocking more characters and worlds to explore — the steps-grow-your-pet precedent)

Core Tech Requirements

Legend: ✓ Applies directly | ♦ Applies with domain adaptation | ● Optional | ✕ Not needed | » Second trimester

Core Tech Requirement	Pawgress
External Database	✓
Authentication »	✓
Analytics »	●
Real-time C++ Simulation Loop and/or C++ Performant Service	✓
Application Communication	✓
Continuous Integration	✓
Data-Driven Architecture	✓
Front-end + Back-end	✓
Custom Tools Development	♦
2D Graphical Output	✓
3D (strong students only)	✕
Networking System	●

How each requirement applies:

Platform fit: **Main:** Native mobile (Native Android — device step-counter + GPS sensors; device: Pixel 9a) — the Native PC build is the development/tools vehicle (the content editor + the debug tools). **Extended:** Firebase + a Linux server the team sets up by M2 (full use T2) for accounts, the cloud store, and the offline-sync backend.

Front end: The animated pet avatar (growth/mood/customisation), the custom-rendered pan/zoom **map of Singapore with fog-of-war exploration**, and the daily-goal/streak/reward UI. **Languages:** Native — Kotlin Native Android UI + the C++ engine via JNI (Android Studio); OpenGL ES for the

realtime 2D (pet + map) in a GLSurfaceView; Web/desktop — SDL2/3 + OpenGL ES for the PC dev/tools build.

Back end: The local C++ loop (pet simulation + animation, the fog-of-war exploration update from GPS, step ingestion, the local UI/graphics update) + the offline save/sync layer with merge/conflict resolution; plus accounts and the cloud store. **Languages:** C++ (loop + sync merge); Firebase-backed services on the Extended tier.

Backend scope: Local now, server-ready (T2) — the accounts / cloud-store / offline-sync seam set up & basic-tested by M2, full use in T2.

- *Loop and/or service:* the C++ loop is load-bearing **once adjustment 2 holds** — it simulates and animates the pet (growth/mood/reaction), updates the fog-of-war exploration state from GPS, ingests step data, and drives the local UI/graphics; the offline→online sync merge is real C++ logic.
- *Data-driven:* pet growth stages + customisation items, map regions/landmarks + exploration rules, and reward/milestone/streak rules are all data — a new pet, region or reward is a data change (authored via the new content editor).
- *2D:* the animated pet and the **custom pan/zoom map with fog-of-war** + the goal/streak/reward UI are real 2D rendering (custom-rendered) — this is the 2D gate. *3D:* not needed. *Networking:* offline save + backend sync is async client↔cloud (Application Communication ✓), so a bespoke Networking System is optional (●).
- *Custom tool:* ♦ the debug tools (GPS spoofer, data injector, network inspector) are supporting infrastructure; **add a content-authoring editor** (regions/landmarks, pet stages/items, reward rules) as the graded tool (see adjustment 1).
- *External DB:* accounts, pet/customisation ownership, exploration + streak progress; local DB for offline. *Auth »:* accounts. *Analytics »:* optional — activity/retention trends.

Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the Games pool in [milestone-goals-pool-per-category.pdf](#) (G1–G3 ★ common, G4+ picks).

M1:

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic**

Tech & dependencies

- **Tech: Web build:** SDL2/3 window + a C++ loop that simulates the pet, updates local state and (on desktop) ingests injected step/GPS data for tooling/tests; C++17/20, CMake, Git. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI, device step-counter + GPS); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** None — foundation of this project.
- **Feeds into:** Every other goal; the pet sim, the map fog-of-war and the sync run on its tick.

- **G2 ★ Data-Driven Architecture of Core Objects / Entity System**

Tech & dependencies

- **Tech:** Entity/component model with pet growth stages + customisation items, map regions/landmarks + exploration rules, and reward/milestone/streak rules in hot-reloadable JSON — a new pet, region or reward is a data change.
- **Prerequisites:** G1 (loop & entity hooks).
- **Feeds into:** M2G2 (the content editor edits this data), M2G13 (progression), M1G14 (region data).

- **G3 ★ Debugging, Profiling, Stress Test & CI** — home of the three named debug tools.

Tech & dependencies

- **Tech:** ImGui debug overlays + the **GPS spoofer**, the **tracking-data injector** (steps/distance) and the **network-break inspector** (client-sent vs server-accepted); Tracy profiling of the pet/map update budget;

Catch2 tests for the sync-merge and data loading; GitHub Actions Windows+Android matrix.

- **Prerequisites:** G1 (loop to instrument).
- **Feeds into:** M2G3 (CI hardening); testing every location/step/network feature deterministically.

- **G4 2D Graphics** — the animated pet avatar + the custom map render (with fog-of-war) are the whole read of the app.

Tech & dependencies

- **Tech: Web build:** Sprite renderer for the pet (growth/mood/customisation states, sprite animation) and a pan/zoom map of Singapore with a fog-of-war overlay revealed by exploration; SDL2/3/OpenGL ES, GLSL, stb_image, FreeType. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** G1 (loop & window), G14 (map data).
- **Feeds into:** M2G4 (advanced pet/map/reward VFX); every visible frame.

- **G12 Save / Persistence & Firebase Integration** — the **offline save & sync** (local store + backend sync with merge).

Tech & dependencies

- **Tech:** Local serialization of pet/exploration/streak state, a sync layer that queues changes offline and merges/resolves conflicts on reconnect, and Firebase (Firestore) for the cloud store; step/GPS data persisted locally first.
- **Prerequisites:** G2 (state to persist).
- **Feeds into:** M2G8 (accounts + cloud sync matured); the offline-first guarantee.

- **G14 Level / Tile Map System** — the explorable **map of Singapore**: chunked/streamed tiles, pan/zoom, and the explored-region data driven by GPS.

Tech & dependencies

- **Tech:** A tile/region map (chunking, pan/zoom camera, spatial indexing) with an exploration mask updated from GPS position, and region/landmark metadata from data.
- **Prerequisites:** G2 (region data), G1 (GPS ingest).

- **Feeds into:** M1G4 (what it renders); the exploration-progress feature; the content editor authors regions.

M2:

- **G1 ★ Architecture — Optimization & Platform Validation**

Tech & dependencies

- **Tech: Web build:** Tracy profiling of the pet/map/sync loop, ASan on MSVC, memory pools, and on-device validation (Pixel 9a) incl. real device step-counter + GPS + offline→online sync; the Firebase/Linux-server seam set up (account + software + basic connectivity test) by M2. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** All M1 goals (hardens the M1 codebase).
- **Feeds into:** The Native-Android deliverable + a robust offline-first sync.

- **G2 ★ Content Editor — Core (Content Authoring Tool) — the required content editor:** author map regions/landmarks, pet growth stages + customisation items, and reward/milestone/streak rules as data.

Tech & dependencies

- **Tech:** Dear ImGui + the shared C++ renderer: place map regions/landmarks, define pet stages + customisation items, and author reward/milestone/streak rules, with round-trip JSON serialization, live preview and undo/redo.
- **Libraries/Software:** Dear ImGui, nlohmann/json, the project's SDL2/3/OpenGL ES renderer (preview).
- **Prerequisites:** G2 (data model), M1G4/M1G14 (renderer + map).
- **Feeds into:** All content; the "extend to more content later" goal; the custom-tools showcase.

- **G3 ★ CI — CMake, Code Quality, Build Standards & Packaging**

Tech & dependencies

- **Tech: Web build:** CMake presets, clang-format + clang-tidy gates, reproducible Windows + Android Debug builds, tests (sync-merge, progression, data loading) and packaging via GitHub Actions. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via

JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.

- **Prerequisites:** G3 (CI & test foundation).
- **Feeds into:** Release/showcase builds of every goal.

- **G13 Gameplay Systems II — the virtual-pet progression:** EXP, growth, milestones, rewards, customisation, and the daily-goals/streaks loop.

Tech & dependencies

- **Tech:** C++ systems converting walking + milestones into pet EXP/ growth, an unlock/customisation economy, and a daily-goal + streak loop — designed to serve the user's wellness, not dark-pattern (data-driven).
- **Prerequisites:** M1G2 (pet/reward data), M1G4 (pet visuals).
- **Feeds into:** The pet-progression + streak features at showcase; T2 content expansion.

- **G8 Firebase — Accounts, Leaderboards & Analytics** — accounts, the cloud store and optional analytics for the offline-first sync.

Tech & dependencies

- **Tech:** Firebase Auth accounts, Firestore for pet/customisation/ exploration/streak data synced across devices, and optional Analytics, via the REST API from C++.
- **Prerequisites:** M1G12 (offline-sync layer).
- **Feeds into:** Cross-device continuity; the sync backend.

- **G4 2D Graphics — Advanced** — polish: pet reward/level-up VFX, map exploration-reveal effects and UI motion.

Tech & dependencies

- **Tech:** OpenGL/GLSL effects and a pooled particle system for reward/ level-up/exploration-reveal moments, plus map polish (region highlights, landmark reveals).
- **Prerequisites:** M1G4 (base renderer), M2G1 (optimization).
- **Feeds into:** The delight/reward moments at showcase.

Track Goals & Team Composition

Team composition (set by instructor): 6 CS + 1 Design + 0 Art (7) — Pawgress lives on **charm** (the pet — growth stages, moods, expression, a customisation-item catalogue — plus the Singapore map/landmarks and the reward art), which is a large art surface that now has **no artist**, so the single designer owns the progression/reward economy and the exploration/retention design *and* takes the **D11 conditional** as a real graded Design goal each milestone (M1 Art Proof of Concept / M2 Art Implementation Quality — the last Design goal each milestone), which consumes one Design+UI slot per milestone (rules: min 1 Design mandated, max 9 total, $D+A \leq 3$; see [team-formation.pdf](#)). **Art-risk flag:** a pet-charm app is very art-forward (pet states + customisation + map/landmarks + rewards), so 0 artists is a serious risk — scope the pet/customisation set hard and lean on a reusable cosy style under D11, or reconsider adding an artist within the $D+A \leq 3$ cap (a solo designer + 0 artists is the case the design pool flags, and squeezes the designer's retention/balance goals). *Scope note (advisory):* pet sim + fog-of-war map + offline sync + progression economy + a content editor is a healthy but full scope — the pet + one explorable map area + the step→EXP→reward loop + offline-sync + the content editor is the M1/M2 floor; broad customisation, many regions and analytics are the stretch.

Track goals — suggested Design / UI-UX / Art / TEAM picks for this project; deliverables & quality ladders live in the track pool documents.

- **Design M1:**

- M1D1 ★ Features Design Document (the walk → pet-grows → explore loop, the reward/streak economy and the wellness framing; primary benchmark e.g. Walkr or Wokamon, with a Singapore benchmark — Healthy 365 — for the local-awareness criterion)
- M1D2 Core Gameplay Loop Prototype (a greybox proving walk → EXP → pet grows / map reveals feels rewarding) · prototype in Unity
- M1D11 Art Proof of Concept *(conditional — no artist; Design-owned)* — the cosy visual direction: the pet (growth stages, moods, customisation-item look), the Singapore map/landmarks and the reward art; 2 mockups. *(Replaces the M1D9 retention-design slot the artist's absence consumes.)*

- **Design M2:**

- M2D1 ★ Features Design Iteration & Showcase Readiness
- M2D11 Art Implementation Quality *(conditional — no artist; Design-owned)* — the pet (states/customisation), the map/landmarks, the rewards and the UI implemented in-engine, no placeholders.

- **UI/UX M1** (graded by the Design instructor for the Games category):

- M1U1 ★ UI Standards & Wireframe Baseline — readability standards for the pet-care home, the map/exploration view and the goal/streak HUD.

- **UI/UX M2:**

- M2U1 ★ Functional UI & Usability — home, map, pet-care/customisation and goals screens navigable unaided on the Native-Android target.
- M2U2 ★ UX Validation & Final Polished UI — validated, polished pet/map/reward UX tested with walkers on the Pixel 9a.

- **Art:** 0 artists — the large art surface (the pet across growth stages + customisation, the Singapore map/landmarks, and the reward art) is covered by the D11 conditionals, taken as Design goals by the single designer (see Design M1/M2). A pet-charm app is very art-forward, so 0 artists is a serious risk — scope the pet/customisation set hard and lean on a reusable cosy style.

- **TEAM M1:**

- M1TA1 ★ Overall Audio Experience — initial BGM/SFX from the week-5 needs list present & working in the prototype (cosy BGM, step/goal-met, pet-happy, reward and region-reveal cues), suitable and correctly triggered.
- M1TA2 ★ Audio Engine Functionality — data-driven audio pipeline (add/replace files without recompiling) + reliable multi-channel playback.

- **TEAM M2:**

- M2TA1 ★ Overall Audio Experience — standard Games floor (cosy ambient BGM + step / goal / streak / pet / reward / region-reveal SFX, ≥6 types), calm and non-nagging in keeping with the wellness tone.

- M2TA2 ★ Audio Engine Functionality — audio properties de/serialised and the full playback functionality the build needs (multi-channel mixing; a distinct reward/level-up cue).
 - M2TP1 ★ Final Presentation — 7 min in front of class + instructors: live custom-tools showcase (the content editor), prototype demo, team post-mortem (wk14)
-

*Generated by the Project Proposal Assessor skill · grounding: live folder
(Project 2 Technical adjustments/) · references verified 2026-09-21.*